A

KNOT

PHILOSOPHY

KNOTTY CLASS

Prakash Hegade



An AI-First classroom preparation guide for Educators

Knotty Class

Author: Prakash Hegade
Published: October 2026
Contact: prakash.hegade@gmail.com
Website: https://prakashbh.github.io/KNOT/
© 2026 Prakash Hegade.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

Suggested Citation:
Hegade, P. (2026). Knotty Class. Available at: https://prakashbh.github.io/KNOT/

"Learning is tied to a problem, tried through thinking, and retied when understanding travels into what comes next."



Prakash Hegade

https://in.linkedin.com/in/prakash-hegade-16879917

Foreword

Research must lead to something meaningful, something that goes beyond publications and finds its place in practice. This book is one such outcome of my research journey. It brings together ideas, experiences, and reflections that I hope will help teachers rethink and refine their classroom practices. The intention is not to prescribe how teaching should happen, but to offer perspectives that encourage teachers to question, experiment, and discover what works in their own classrooms. If this book helps even a few teachers see their classrooms differently, the research behind it will have served a meaningful purpose.

This book is written in the first person because its ideas grew from my own teaching, research, and classroom experiences. I hope this voice brings you closer to the journey and invites you to reflect on how its ideas might take shape in your own classroom.

CHAPTER 1

INTRODUCTION

As a teacher, I have always looked forward to the next class with excitement about what new thing I could try. Across more than fifteen years of teaching, each class has offered an eye-opening moment: a new way to ask a question, invite curiosity, or help students become more inquisitive. I kept coming back to the same questions: How can I make my class more curious? How can I encourage students to investigate, rather than simply receive, what I teach?

KNOTTY CLASS grew from those questions and from a longer research journey. The work did not begin in a single day. It's most concentrated stretch began around 2019, when I completed the Problem-Based Learning certification *One Day One Problem* from Singapore Polytechnic. That experience excited me and gave shape to ideas I had been exploring in my teaching. Later that year, I published a paper on *One Day Many Problems*, an approach that I felt fitted the Indian context. I continued learning, reading, experimenting, and reflecting. A

course on Problem-Based Learning from Aalborg helped me understand the philosophy more deeply and appreciate its scope.

Over time, the exploration became a study of how classroom experiences could support students as they made sense of ideas. I tried activities, cases, projects, and different ways of asking students to think. Model-thinking workshops gave me opportunities to experiment with aspects of Problem-Based Learning and Computational Thinking. Student work and feedback offered more perspectives. Each attempt raised new questions. What made a problem worth engaging with? How could students show their thinking? What might help them continue when an idea was unclear? How could they carry what they had learned into a different situation?

This book shares what I learned through that process in a form teachers can use. It introduces the KNOT framework as a practical guide for designing classroom experiences. KNOT means "KNOT is Not One-time Transformation". The name is recursive by design. Learning is not a single change that happens once and then ends. What students understand can shape the next question; a new problem can test an earlier idea, extend it, or invite students to build their understanding again.

The philosophy behind the framework is "**Tied, Tried, Retied**". Learners tie ideas to a meaningful problem, try those connections through thinking and evidence, and then retie them as they carry understanding into another context. A knot is not a final answer. It is a connection that can be tested, loosened, strengthened, and made useful in what comes next.

The important insight for me was that a meaningful problem gives learning a beginning, but the deeper work lies in what students do with it. They represent it, explain a choice, inspect what happens, and connect the result to a principle. A problem can create interest, but interest alone does not show what a learner understands. Students need opportunities to reason, make their thinking visible, receive feedback, revise an explanation, and try an idea in a new setting. The teacher's role is to notice what their thinking shows and respond in ways that help them take the next step.

KNOT turns this belief into four flexible teacher moves: Kick off, Navigate, Observe, and Tie back. Kick off gives learners a reason to begin, often through a question, problem, case, or challenge. Navigate helps them work through it by representing information, breaking the

problem into parts, identifying patterns, comparing options, and explaining their decisions. Observe asks the teacher to attend to student thinking as it appears in questions, attempts, discussions, solutions, and assessment. Tie back helps students connect their learning to concepts, applications, and questions that can travel beyond the class.

These moves are not four rigid stages or a script that every lesson must follow in the same way. They are prompts for designing and responding. A short class activity, a programming lab, a case study, and a longer project can each use them differently. Some lessons may move quickly; others may need more time for students to explore, practise, and reconsider. The framework is meant to help teachers make purposeful choices while remaining attentive to what their learners need.

KNOT brings together ideas that informed my research. Problem-Based Learning provides a meaningful context for inquiry. Computational Thinking offers ways to reason through problems, including decomposition, pattern recognition, abstraction, and algorithmic thinking. Liminality helps a teacher notice that students may experience uncertainty or partial understanding while working

with difficult ideas. Dialogue, practice, scaffolding, and feedback can help students continue. These ideas do not guarantee one outcome for every learner. They help teachers think carefully about how a learning experience is designed and how it can respond to evidence of student thinking.

The framework emerged from classroom work and the evidence gathered in my study. Students described taking initiative, working through problems, connecting concepts, and applying ideas. Their accounts, project evidence, course records, and survey responses helped me see knowledge construction as more than recalling information. It can involve organising ideas, explaining relationships, making decisions, testing approaches, and carrying understanding into another context. These findings belong to the setting in which they were gathered. KNOT offers a way to design and examine learning; teachers can adapt it thoughtfully to their learners, subjects, and classrooms.

KNOTTY CLASS is an invitation to do that. It is not a collection of perfect lessons to copy. It is a companion for asking better planning questions: What will give learners a reason to begin? What thinking do I want them to practise? How will I notice what they understand? Where

can they use the idea again? The chapters that follow explore these questions and offer a planning template, examples, and ways to bring KNOT into everyday teaching.

My central belief is simple: understanding is not a finish line. It is a connection we make, examine, and carry into what comes next. When a class helps students do that for themselves, learning can continue beyond the lesson.

CHAPTER 2

THE EMERGENCE OF KNOT

KNOT took shape in two distinct moments. The pilot study produced an early framework, KNOT Version 0. The official study then gave me a wider body of evidence with which to examine, refine, and express the final framework. Keeping these moments separate matters. Version 0 records what the pilot made possible; the KNOT presented in this book reflects the learning that followed.

The pilot was a period of trying things out more systematically. I examined classroom activities, gathered feedback, and explored possible ways of understanding students' learning. Some survey questions needed clearer wording. Some analyses helped me understand the course but did not need to carry forward into the main study. The pilot's value was partly in showing me what to reconsider. KNOT Version 0 emerged from this stage as an initial way to organise the ideas. It was a starting point for the next study, not the final account of what KNOT would become.

The official study involved a different student batch. In a Design and Analysis of Algorithms course, I brought together Problem-Based Learning and Computational Thinking through the way concepts were delivered, the cases and lab exercises, and the project work. I examined several kinds of evidence: survey responses, students' written reflections, project work, concept-understanding responses, and course records. Each source offered a different view of the learning experience. Student accounts helped me understand how learners described their thinking; their work and course records showed other ways that learning was expressed.

As I read across these sources, I kept returning to the study's central concern: how knowledge construction was supported and evidenced. Self-directed learning, problem solving, and transfer of knowledge became useful learning dimensions through which to interpret that concern. Students' accounts included examples of practising on their own, breaking a problem into parts, connecting concepts, and applying an idea in a project context. These patterns did not appear as one identical path followed by every learner. They helped me see the different ways students could engage with and use what they were learning.

The final framework emerged when I brought the course design and the evidence into conversation. The teaching materials showed the opportunities students were given. Their responses and work helped me consider what those opportunities made possible and how students experienced them. I began to see that a framework for teachers needed to hold both sides of the classroom experience: the decisions teachers make and the thinking learners undertake. KNOT gives that design a simple form through four teacher moves: Kick off, Navigate, Observe, and Tie back.

The name carries the idea at the centre of the framework: KNOT is Not One-time Transformation. It is a recursive definition. Learning can return to an earlier idea with a different question, a new application, or a more developed understanding. A student may revisit what they thought they knew and reshape it. A teacher may use what students show in one part of a lesson to decide what to do next.

KNOT is the final framework developed through this research. It gives teachers a way to think about planning classroom experiences while leaving room to adapt them to their students and subject. The work that follows in this book explains how to use those moves and what they can help a teacher notice.

If you want to understand the complete research, it is here: https://prakashbh.github.io/KNOT/

CHAPTER 3

WHAT KNOT IS DESIGNED TO SUPPORT

A framework becomes useful when it helps a teacher make a classroom intention visible. KNOT is designed to support three related learning dimensions: self-directed learning, problem solving, and transfer of knowledge. These dimensions work together and contribute to a broader process: knowledge construction.

They are not three separate outcomes to tick off at the end of a lesson. A student may take initiative while trying to solve a problem. In doing so, the student may connect an idea to something learned earlier or use it in a new context. These actions can show how knowledge is being built, tested, and made useful.

Self-directed learning: taking initiative with support

Self-directed learning does not mean that students must learn alone or that the teacher steps aside. It means students have opportunities to take responsibility for parts of their learning: to ask questions, recognise what they need to find out, choose a way to proceed, and check their own understanding.

A class can support this by giving students a clear purpose and room to make decisions. They may need to decide what information matters, which resource to use, what to practise, or when to ask for help. A teacher can provide a useful starting point and still leave meaningful work for students to do.

Support matters here. A learner who is unsure may need a prompt, an example, a conversation, or feedback before being ready to continue independently. The intention is not to remove guidance. It is to offer guidance that helps students take the next step for themselves. In the study, students described actions such as practising, seeking explanations, and working through ideas with others. These accounts help show what self-direction can look like in a course. They also remind me that independence is not always quiet or solitary. Asking a peer to explain an idea, looking for another example, or returning to a difficult concept can all be ways of taking responsibility for learning.

Problem Solving: making decisions and explaining them

Problem solving involves more than reaching an answer. Students need to understand what a problem is asking, identify relevant information, represent the situation, and decide how to work through it. They may

break it into parts, notice patterns, compare possible approaches, and check whether a solution makes sense.

This is particularly important in computer science (also other domains). Students may be able to repeat an algorithm or write code that works for one example, yet still be unsure why it works or when it would be useful. A problem-solving experience gives them a reason to examine the choices behind a solution.

The teacher can ask students to explain how they interpreted the problem, what assumptions they made, why they selected a particular structure or method, and how they checked their answer. These questions make reasoning available for discussion.

They also help the teacher see whether a student needs further explanation, a smaller example, or another opportunity to try. In the course materials, students encountered problems that called for decisions about data, routes, constraints, and efficiency. The point of using such situations was not simply to make an exercise look familiar. The situation gave students something to reason about. Their task was to connect the context to a representation and then justify how they approached it.

Transfer of Knowledge: carrying an idea into another context

Transfer happens when students recognise that an idea learned in one situation may help them understand or solve a different one. A new context may look different on the surface, so students have to identify what is similar, decide which concept applies, and adapt their understanding.

A real-world setting by itself does not demonstrate transfer. Students need to show how the course idea connects to the situation. They might explain why a graph representation fits a route-planning problem, or how a data structure supports repeated searches. The explanation matters because it shows the connection the student has made.

Projects can provide space for this kind of work. In the study, students described using course concepts in project decisions and relating algorithms to real-world systems. A student who applies graph theory to plan connected roads, for example, is doing more than naming a topic. The student is connecting a formal idea with a practical representation and explaining its usefulness. Not every student will make the same connection, and not every application will be equally

developed. The teacher can help by asking students to identify which idea they used, how it fits the new situation, and what had to change when they applied it.

Knowledge Construction: bringing the dimensions together

Self-direction, problem solving, and transfer help describe different parts of students' learning. Knowledge construction is the wider process that brings them together.

A student may begin by deciding what to investigate. While working on the problem, the student may organise information, test an explanation, or revise an approach. Later, the student may connect the idea to a project or another situation. Across these actions, knowledge becomes more than information remembered for a single question. It becomes a set of relationships the learner can explain, examine, and use.

This process may include uncertainty. A learner may have a partial explanation, try it, notice something does not fit, and reconsider. The teacher's task is to notice what the student can show at that point and provide a useful next opportunity. The framework values this

development because understanding can be incomplete before it becomes clearer.

The study examined student reports alongside their work and course records. Each source offered a different view. Survey responses showed what students reported; written accounts helped describe their experiences; projects and course records offered evidence of how they used or demonstrated learning.

Read together, these sources helped me understand how the three learning dimensions related to knowledge construction in this course.

Table 3.1: Learning Dimensions and Teacher Look for

Learning Dimension	What a Teacher Can Look For
Self-Directed Learning	Students identify what they need to learn, practise, seek resources or support, and check their understanding.
Problem Solving	Students represent a problem, select and justify an approach, examine constraints, and evaluate their solution.

Transfer of Knowledge	Students connect a course idea to a different context and explain how they adapted it.
Knowledge Construction	Students develop, connect, test, and revise ideas across these actions.

This relationship is not a guaranteed sequence. A student may move between the dimensions, return to an earlier idea, or need support at different points. The framework helps teachers design opportunities for these kinds of learning and pay attention to evidence as it appears. It does not promise that every student will respond in the same way.

For me, the central value of KNOT is that it keeps these learning dimensions connected. Students need a chance to take initiative, a worthwhile challenge to reason through, and opportunities to carry ideas further. When these experiences come together, teachers can better see how students are constructing knowledge and decide what might help them continue.

CHAPTER 4

THE KNOT'S PHILOSOPHY

A belief about learning shaped by years of teaching, classroom experiments, and listening to students make sense of difficult ideas. After years of teaching, one thing has stayed with me: a class becomes memorable when students have a reason to wonder, a way to work on the question, and room to change their minds.

Tied, Tried, Retied names the philosophy I draw from this study: learners tie ideas to a meaningful problem, try those connections through thinking and evidence, then retie them as they carry understanding into a new context. A knot is not a final answer. It is a connection that can be tested, loosened, strengthened, and made useful in what comes next.

"Learning is tied to a problem, tried through thinking, and retied when understanding travels into what comes next."

Learning is the work of making and remaking connections. For many years, I entered each class wondering what might make students

curious that day. Each class offered another chance to notice what they understood, where an idea became difficult, and what might help them look again. This study grew from that question: how can a class invite students to think, not only receive what is already known?

The work has led me to believe that knowledge is not simply delivered. Students build it as they connect a new idea with what they know, use it to make sense of a problem, and check whether their explanation holds. A connection may be strong or incomplete. A new problem may show that an earlier understanding needs to be opened up and made again. That is part of learning.

A meaningful problem gives this work purpose and a reason to ask questions. Computational thinking offers ways to work through it: break it into parts, notice patterns, decide what matters, develop a procedure, and check whether it works. These practices help students give shape to their thinking so they can discuss, test, and improve it.

The case and lab sheets made this belief more concrete for me. The Hungry Loop Cafe case began with a service that slowed down when it repeatedly searched its records. The Invitation Problem began with visits students could imagine, then asked them to make decisions

about routes, cost, time, assumptions, and a general algorithm. One of the labs moved from familiar activities and shared expenses to programming and the comparison of possible solutions. I now see that a problem gives learning a beginning, but the deeper work is in how students represent it, explain a choice, inspect what happens, and connect the result to a principle.

Students also need room to be uncertain. Their accounts included effort, repeated practice, connecting topics, seeking help, and explaining ideas to peers. This does not mean struggle automatically produces learning. Students may need time, resources, conversation, feedback, and a safe opportunity to try again. A teacher's work is to notice when support can help a learner take the next step.

This is what I mean by Tied, Tried, Retied. Learners bring existing ideas into a new situation. A problem may strengthen a connection, reveal a gap, or bring separate topics together. The learner tests an explanation, revises it, and carries some understanding forward. Another context may ask for a new connection. Learning continues because what students know can become more useful, connected, and open to question.

Students' project feedback helped me recognise this connection-making in the study itself. Many reported using class concepts in the project and seeing algorithms in real-world systems. In one open response, a student described applying graph theory to plan connected city roads. I read this alongside the submitted portfolios: they show what such application could look like in route planning, emergency services, resource allocation, and other city systems. The survey and portfolios do not prove that every student learned in the same way, but together they show why application and explanation belong in the philosophy of knowledge construction.

KNOT brings this belief into course design. Problem-Based Learning gives students a meaningful context; Computational Thinking helps them reason through it. Attention to liminality helps a teacher recognize unsettled understanding. Scaffolding, dialogue, practice, and feedback help students continue. Together, these elements can create conditions for knowledge construction, but they do not guarantee the same result for every student or form a fixed recipe.

I hope this work offers teachers permission to design with intention and remain responsive. Begin with a question worth exploring. Make students' thinking visible. Let them test and explain ideas. Offer

support, and leave space for a new question. The final KNOT framework can be examined in other settings and adapted thoughtfully to local learners and subjects. Its value is in helping us ask better questions about how learning is designed and experienced.

My central belief is simple: understanding is not a finish line. It is a connection we make, examine, and carry into what comes next. When a class helps students do that for themselves, learning can continue beyond the lesson. The work of teaching is not to tie every idea for a student. It is to create the conditions in which they can make, test, and remake the connections themselves.

CHAPTER 5

KNOT FRAMEWORK

KNOT Framework is a practical guide for designing PBL–CT classroom experiences. Its guiding philosophy, Tied, Tried, Retied, describes how learners connect ideas to a problem, test their thinking, and carry prior learning into what comes next.

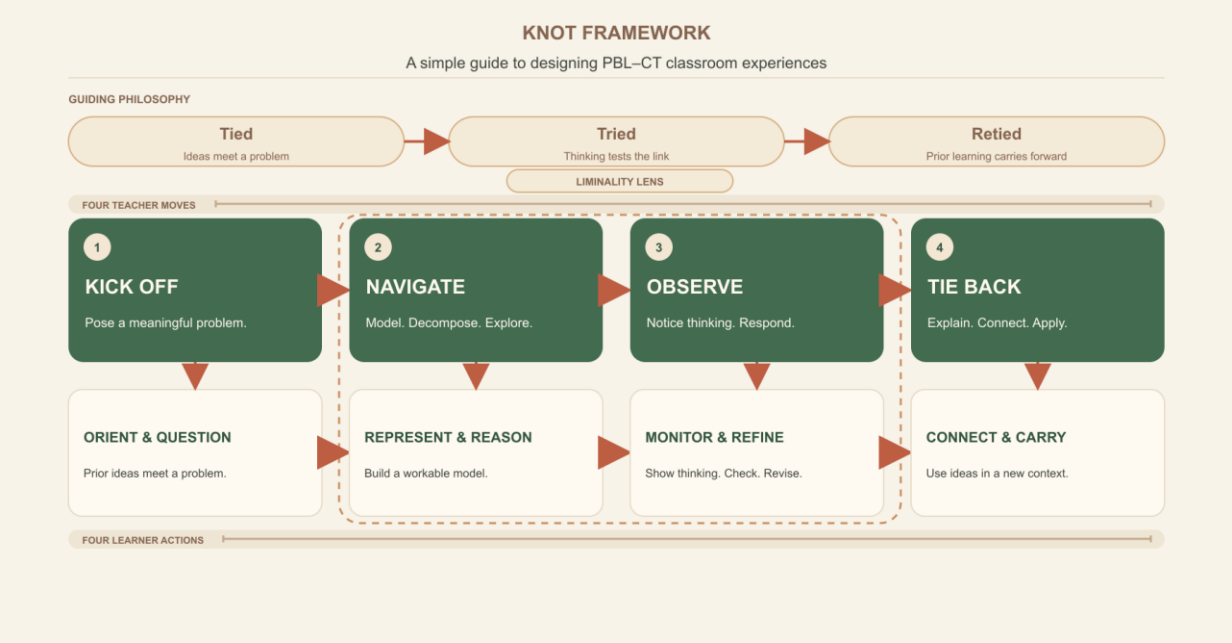


Fig. 5.1: KNOT Framework

The framework brings together four teacher moves: Kick off with a meaningful problem, Navigate through modelling and exploration, Observe student thinking and respond, and Tie back by inviting

learners to explain, connect, and apply their ideas. Beneath each move, the diagram shows a corresponding learner action, from orienting to a question through to carrying ideas into a new context. The Liminality lens highlights that uncertainty and partial understanding can arise during learning.

The diagram shows how the KNOT Framework helps a teacher plan a learning experience and notice what students are doing as they learn.

Kick-off: The teacher poses a meaningful problem. Students connect it with what they already know and begin asking questions.

Navigate: The teacher helps students explore and work through the problem. Students build a model, break the problem into parts, and reason through possible approaches.

Observe: The teacher looks for evidence of student thinking and responds with guidance or feedback. Students show their thinking, check their ideas, and revise when needed.

Tie back: The teacher invites students to explain and apply what they have learned. Students connect the idea to a new context and carry it forward.

The top row, Tied → Tried → Retied, expresses the guiding philosophy: learning connects ideas to a problem, tests those connections through thinking, and carries prior learning into what comes next.

The liminality lens draws attention to moments when understanding is still developing. The framework supports a learning process that can continue across questions and contexts, rather than ending with one activity.

The KNOT components can be seen below in Figure 5.2.

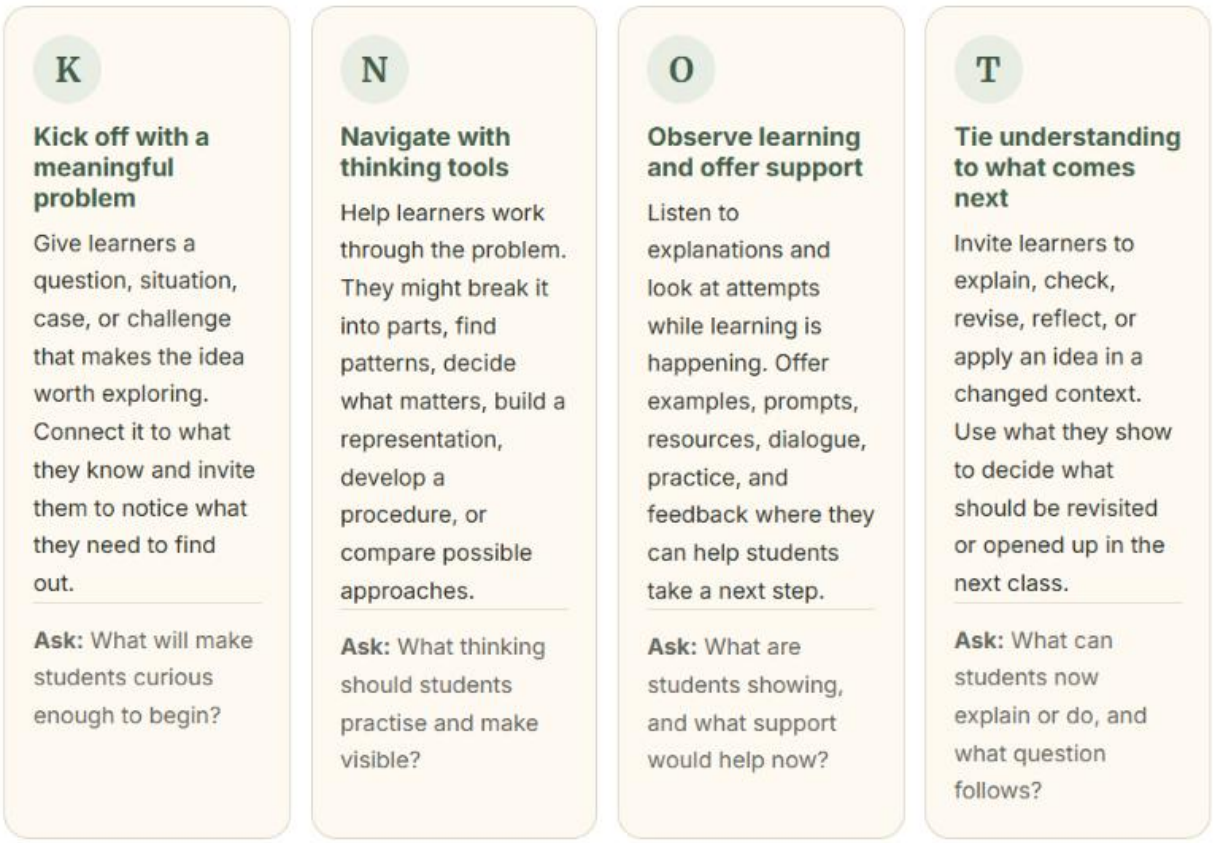


Fig. 5.2: KNOT Components

CHAPTER 6

KNOT CLASS EXAMPLES

The next three examples show how a KNOT class could take shape across different areas of computer science. The first uses Depth-First Search from Algorithms, the second explores threads from Operating Systems, and the third introduces TCP from Networking. Each begins with a classroom hook and follows the KNOT moves in its own way, showing how the framework can be adapted to the concept being taught.

KNOT Class Plan: Depth-First Search

Duration: 60 minutes

Format: Story, group exploration, and paper-based tracing. No coding.

The hook: The old fort rescue

Begin with this story:

A storm has cut the power in an old hill fort. A caretaker is trapped somewhere in its underground chambers. The rescue team has a map showing which chambers are connected, but they do not know where the caretaker is. They must explore the passages, mark each chamber they enter, and avoid checking the same room twice.

Your team will decide how to search.

Show students the map, but do not name DFS yet. Ask: “If you enter a chamber and find more than one passage, how will you choose? What will you do when every passage from that chamber has already been checked?”

Learning goals

By the end of class, students should be able to trace DFS, explain how it uses marking and backtracking, and recognise that it does not necessarily find the shortest route.

Time	KNOT move	Teacher and student activity
0–7 min	Kick off	Tell the rescue story and show the chamber map. Students suggest search rules. Invite them to consider how the team can avoid revisiting a chamber or getting stuck at a dead end.
7–22 min	Navigate	Give groups chamber cards and connected strings or a printed map. One student acts as the rescuer. Starting at A, they choose an unvisited connected chamber and mark each room they enter. When there are no unvisited passages, they retrace their route to the most recent chamber with another passage to

		explore.
22–34 min	**Navigate**	Introduce the name Depth-First Search after students have experienced the method. On paper, groups record the order of rooms and use a stack to track the active route. Ask them to explain what goes onto the stack and when they backtrack.
34–46 min	**Observe**	Give groups a second starting point or a changed connection. Ask them to predict the next room before tracing it. Look for repeated visits, skipped neighbours, and confusion about when to backtrack. Have students explain and correct their traces.
46–60 min	**Tie back**	Reveal that the caretaker is in chamber F. Compare the team's DFS route with the shorter route from A through C to F. Discuss why DFS can find a target without finding the shortest path. Students complete the exit prompt below.

Exit prompt

Ask each student to answer:

1. Where did the search backtrack, and why?

2. What helped the team avoid revisiting a chamber?

3. Did DFS find the caretaker by the shortest route? Explain.

This class lets students feel the logic of DFS before learning its name: follow one path deeply, keep track of where you have been, and backtrack when that path has no further unvisited option.

KNOT Class Plan: Threads in an Operating System

Duration: 60 minutes

Approach: Concept-first, with a story and classroom simulation. No coding.

Learning goals

By the end of class, students should be able to explain:

- How a thread relates to a process.
- How threads in the same process can share data.
- How a race condition can occur.
- How a lock can protect a shared resource.

The hook: Two people book the last seat

Start with this situation:

There is one seat left on a bus for a college trip. Two students press Book at almost the same time. The booking system handles both requests. How could it accidentally issue two tickets for that one seat?

Ask students to work out what each request might see and do. Let them make predictions before introducing the terms process, thread, or race condition.

Time	KNOT move	Teacher and student activity
0–8 min	Kick off	Present the last-seat story. Students individually write down how two bookings might both succeed, then compare their explanations with a partner.
8–26 min	Navigate	Run the booking simulation. Two students act as request-handling threads. They take these steps: read available seats, check whether a seat is available, update the count, and confirm the booking. Use a board or card showing **Seats available: 1**. Let the students interleave their steps.
26–38 min	Navigate	Introduce the process and thread model. Represent the booking application as one process. Show that its threads share the application's data, including the seat count, while each thread keeps its own execution state. Ask students to map the actions from the simulation to this model.
38–50 min	Observe	Repeat the simulation using a **lock token**. A thread must hold the token while it checks and updates the seat count. The other thread waits. Ask students to compare the result with the first round and explain what the lock protected.
50–60 min	Tie back	Move to a new situation, such as two supermarket checkouts updating the stock of the last item. Students identify the shared data, the two threads, the possible race, and where a lock might be needed. Finish with an individual exit response.

What happens in the simulation?

In the first round, the steps might interleave like this:

1. Thread A reads 1 seat.

2. Thread B reads 1 seat.

3. Thread A updates the count to 0 and confirms a booking.

4. Thread B also updates the count to 0 and confirms a booking.

Both threads acted on the same earlier value. The system may issue two confirmations even though only one seat was available. This is a race condition: the result depends on how the threads' operations overlap.

In the second round, the lock allows one thread at a time to check and update the seat count. Thread A books the seat. Thread B then sees that no seats remain. The protected check-and-update sequence is a critical section.

Making the process and thread distinction clear

Use the booking application as the example:

Process	Threads within the process
The running booking application, with its resources and memory	Separate paths of execution handling requests
Holds the shared seat-count data	Can access shared process data
Provides the environment in which	Each thread has its own execution state,

its threads run	such as its stack and current instruction

Exit response

Ask each student to answer:

1. What shared data caused the problem in the booking example?

2. Why could the two threads both believe they had booked the last seat?

3. What did the lock change?

What this KNOT class is designed to reveal: students first encounter a problem, then model how concurrent actions affect shared data. Their explanations show whether they understand the race condition and the purpose of synchronization. They then carry that idea into another situation where multiple threads may update the same resource.

KNOT Class Plan: Understanding TCP

Duration: 60 minutes

Approach: Concept-first, using a story and classroom simulation. No coding.

Learning goals

By the end of class, students should be able to explain:

- Why TCP is used when applications need reliable, ordered data.

- What happens during a TCP three-way handshake.

- How sequence numbers, acknowledgements, and retransmission help recover from loss or reordering.

The hook: Delivering a document through an unreliable courier

Begin with this story:

You need to send a ten-page document to a colleague. The courier may deliver pages in a different order, and one page might go missing. Your colleague needs the complete document in the right order. How could you organise the delivery, and how would you know it arrived?

Give groups page cards with simple numbers. Have one group act as the sender, one as the receiver, and another as the network. The network can reorder or temporarily "lose" a page. Ask students to invent a way for the sender and receiver to coordinate.

Time	KNOT move	Teacher and student activity
0–8 min	Kick off	Tell the courier story. Students discuss what could go wrong when data is sent through a network and propose ways to handle it.
8–22	Navigate	Run the page delivery simulation. Send numbered pages out of

min		order and hold one back. The receiver arranges the pages, identifies what is missing, and sends a message indicating what it still needs.
22–34 min	**Navigate**	Connect the activity to TCP. Explain that TCP establishes a connection, numbers ranges of bytes, uses acknowledgements, and retransmits data when needed. Model the three-way handshake with students acting as client and server.
34–48 min	**Observe**	Give groups a new delivery sequence with reordered and missing pieces. Ask them to track what arrived, what the receiver can acknowledge, and what the sender needs to resend. Check whether students understand that TCP presents the data in order to the receiving application.
48–60 min	**Tie back**	Compare two uses: downloading a document, where complete and ordered data matters, and a live voice conversation, where waiting too long for delayed data may be a problem. Students explain why an application's needs influence its choice of transport protocol.

Model the TCP connection

Use three students or three cards to act out the handshake:

1. Client: "I want to start a connection."
2. Server: "I received your request and I am ready."
3. Client: "I received your response. Let's begin."

Then label the messages SYN, SYN-ACK, and ACK. Explain that this exchange helps the two endpoints establish the connection and synchronise their initial sequence numbers before data transfer.

Model loss and ordering

For a simple classroom model, treat the document pages as numbered pieces of data. Send pieces 1, 2, and 3, but let the network deliver 1 and 3 first while holding back 2. The receiver can indicate that it is still waiting for piece 2. The sender retransmits the missing data, and the receiver can then present the stream in order.

Clarify that real TCP sequence numbers count bytes, not document pages or application messages. The numbered pieces are a teaching model for understanding ordering and recovery.

What to observe

Look for whether students can explain:

- Why the receiver needs sequence information to put data in order.
- How acknowledgements help the sender know what has arrived.
- Why missing data may need to be retransmitted.
- How TCP's reliability can involve waiting, which may not suit every time-sensitive application.

TCP provides a reliable, ordered byte stream while a connection can be maintained. It does not guarantee that communication will succeed if the network or connection fails completely. Flow control and congestion control can be introduced in a later class.

Exit prompt

Ask students to answer in their own words:

1. What problem does the TCP handshake help address?

2. What do sequence numbers and acknowledgements help the endpoints do?

3. Why might an application prioritise timely delivery over retransmitting every missing piece?

The class begins with a delivery problem, lets students build a coordination method, and then connects their model to TCP. Students finish by considering how the communication task shapes what an application needs from the network.

KNOT Gives the Class Its Skeleton

KNOT offers a basic skeleton for planning a class. It helps a teacher think through how to begin, how students might work through the learning, what their thinking could reveal, and how they might carry

the learning forward. It is a starting structure, not a finished lesson plan.

The teacher still needs to shape each move for the learners, subject, time, and classroom context. Begin with the learning objectives, then refine every phase so students have a clear opportunity to work towards them. Consider what students already know, what materials or support they may need, how you will recognise their progress, and how the final activity will show whether the objectives have been met. The more thoughtfully each phase is adapted, the more effectively KNOT can support the class.

CHAPTER 7

DESIGNING YOUR OWN KNOTTY CLASS

Go to any LLM, upload the PDF titled KNOTTY-Class-Framework-and-Philosophy-Guide.pdf. Complete the missing text and give the prompt provided along with it.

Subject or course:

Concept or topic:

Student level or year:

Approximate number of students:

Class duration:

Classroom format: in person, online, or hybrid

Use the uploaded KNOTTY Class Framework and Philosophy Guide and above details to prepare a class plan.

The class plan you receive is a starting point. Refine each phase to suit your learners, learning goals, available time, and resources. As classroom needs change, adapt the plan using your knowledge of the students and the learning you want to support.

CHAPTER 8

WHY KNOTTY CLASS?

A class can be busy without giving students a reason to think. They may complete an activity, follow steps, or receive an explanation, yet still be unsure how an idea works or when it might be useful. KNOT helps me focus on the design choices that make student thinking part of the class: what gives them a reason to engage, how they work through an idea, what their thinking shows, and how they can use what they learn.

A clear purpose for the class

KNOT gives teachers a way to connect a learning objective to a classroom experience. The opening problem or question gives students something to investigate. The work that follows gives them opportunities to represent, reason, test, and explain. The teacher plans how to notice their thinking and what evidence will show progress. The class then gives students a chance to connect or apply what they have learned.

This makes planning more coherent. A teacher can look at a lesson and ask: Does the opening lead into the intended learning? Do students have to think with the concept? How will I see what they understand? Will they have a chance to explain or use the idea? These questions can help identify when an activity is engaging but disconnected from the learning objective, or when an assessment asks for something students have not had a chance to practise.

The value of a hook

A hook is not just an entertaining opening. It gives learners a reason to care about the question and a point from which to begin. A good hook may connect with something familiar, present a puzzle, introduce a need, or show that the current understanding is incomplete.

This matters when students can ask AI for a definition, explanation, or solution almost immediately. If a class begins and ends with receiving an answer, students may have little reason to form an idea of their own or examine how that answer was produced. A purposeful opening can create that reason. Students can make an initial prediction, identify what they do not know, or describe how they might approach the problem before consulting a tool or another source.

The hook gives later work something to return to. Students can compare their first thinking with the evidence they find, explain what changed, and decide whether the answer they now hold is more convincing. Their learning becomes visible across the class, not only in a final response.

Why students need to iterate

Learning often requires more than one attempt. An explanation may seem reasonable until a student tests it with an example. A strategy may work in one situation but fail when a condition changes. A response from an AI tool may sound confident but include a mistake or leave out an important assumption.

Iteration gives students time to examine these differences. They can make an initial attempt, use feedback or evidence, revise their reasoning, and explain why they changed their approach. Revision is then part of learning rather than a sign that the first attempt was wasted.

The teacher's observation matters here. A wrong answer alone may not tell us what a student understands. A trace, a diagram, a choice between strategies, or a brief explanation can reveal more. The

teacher can use that evidence to decide whether students need another example, a question, peer discussion, feedback, or more time.

What is innovative about KNOT?

The individual ideas behind KNOT are familiar in education. Problem-Based Learning gives students a problem to work on. Computational Thinking offers practices for representing and reasoning through problems. Reflection, feedback, and scaffolding help teachers respond to learning.

KNOT's pedagogical contribution is in how it brings these ideas into a usable design structure. Its four teacher moves connect planning with what learners may do: Kick off gives a purpose for learning; Navigate creates opportunities to reason; Observe helps the teacher respond to evidence; and Tie back invites students to explain, connect, or apply their learning. The framework also keeps self-directed learning, problem solving, and transfer of knowledge in view as related dimensions of knowledge construction.

The recursive philosophy of KNOT adds another point of emphasis. What students learn in one class can shape the questions they bring to the next. A class does not have to be treated as an isolated event with

a single finish line. The teacher can consider how learning might be revisited, extended, or used in another context.

I would describe the innovation as a way of bringing design, learner action, and evidence into relationship. It gives teachers a simple structure for making decisions while leaving room for professional judgement and adaptation.

Why this matters in an AI-first world

AI tools can generate explanations, examples, questions, and draft solutions. Students will need to do more than receive those outputs. They need to decide what to ask, check whether a response fits the problem, test its claims, identify what may be missing, and take responsibility for what they use.

For a teacher, the important question is what students do with the help AI provides. Can they explain the problem, examine the response, check it against evidence, and adapt it to the task? KNOT helps teachers plan for those actions. The class begins with a question students can think about; they then explore possible approaches, make their reasoning visible, and connect what they learn to another

situation. AI can support this work when it is useful, while the teacher shapes the learning purpose, guidance, and assessment.

KNOT can also help a teacher use AI when preparing a class. A teacher can provide the subject, topic, learner level, class size, duration, and classroom format. AI can then produce a first draft with a case study, suggested learning outcomes, and a KNOT class plan. The teacher reviews the details and refines the draft to fit the students and the course.

A framework that leaves room for the teacher

KNOT does not produce the same lesson for every subject or guarantee the same result for every learner. It gives the class a starting structure. The teacher decides how to shape each move for the students, learning goals, time, resources, and assessment.

That is why KNOT is useful to me and you. It keeps planning connected to what students are being asked to learn and what they will do with that learning. It helps teachers create a reason to begin, pay attention to thinking as it develops, and make space for students to carry ideas further. In an AI-rich world, those decisions remain part of the teacher's work.

CHAPTER 9

TEACHING TODAY

Students can now ask an AI tool to explain a concept, suggest a solution, or produce a first draft in seconds. The important classroom question is not only whether they can get an answer. It is whether they can understand the problem, judge the response, and explain what they have learned from it.

A polished response can still contain an error, miss an important detail, or fail to fit the task. Students need opportunities to compare a response with evidence, test an idea, and decide what they can use. That work depends on understanding the subject. Without it, a student may have difficulty recognising when an answer needs to be questioned.

This is one reason the opening of a class matters. A meaningful question gives students something to think about before an answer is offered. They can make a prediction, identify what they need to know, or try an initial approach. When they later consult AI or another resource, they have a starting point for examining what it provides.

The next step is to give students room to work with the response. They might ask whether it addresses the problem, test it with an example, find a missing assumption, or compare it with another approach. If their thinking changes, they can explain why. This kind of iteration helps students develop judgement. Repeating a prompt until an answer looks right is not the same as checking whether the answer is sound.

KNOT helps teachers plan these opportunities. Kick off gives the class a purpose and a question to consider. Navigate gives students space to work through the problem, with or without AI. Observe helps the teacher see how students are reasoning and where they may need support. Tie back invites students to explain what they understand and how they might use it elsewhere.

AI can be useful in this process. It can offer another explanation, suggest a possible approach, or help create a first draft. The teacher still decides what the class is trying to learn, what evidence matters, and how students will show their understanding. Students remain responsible for examining and explaining the ideas they use.

This does not mean that every class needs an AI tool. It means that teachers can design learning so students practise thinking with information, checking it, and carrying understanding forward. When answers are instant, the ability to ask, examine, test, and explain becomes even more valuable.

CHAPTER 10

CONCLUSION

A KNOTTY Class is not defined by a particular activity or by the use of AI. It is shaped by the choices a teacher makes: how to give students a reason to begin, how to help them work through an idea, how to notice their thinking, and how to connect learning to what comes next.

KNOT offers a structure for making those choices. Kick off, Navigate, Observe, and Tie back can guide a lesson, a lab, a case study, or a larger project. The moves are flexible. Their value comes from how well they serve the learning goals and the students in front of you.

The class plans and templates in this book are starting points. A generated plan may offer a useful case, suggested outcomes, and a sequence of activities. Your knowledge of the subject and your learners helps you decide what to keep, adapt, or change. During the class, student thinking may lead you to adjust again. That response is part of thoughtful teaching.

KNOT grew from a study of how students engage with problems, develop ideas, and use what they learn. It offers a way to design for those possibilities while leaving room for teachers to make the framework their own.

The work continues after one class. Notice what students carry forward, and let that shape the next question.

Major References

… that shaped this philosophy

Problem-Based Learning

Barrows, H. S. (1986). A taxonomy of problem-based learning methods. Medical Education, 20(6), 481–486. https://doi.org/10.1111/j.1365-2923.1986.tb01386.x

Useful for explaining how PBL can take different forms and why the teacher's design and facilitation matter. (Wiley record)

Threshold concepts and liminality

Meyer, J. H. F., & Land, R. (2003). Threshold concepts and troublesome knowledge: Linkages to ways of thinking and practising within the disciplines. In C. Rust (Ed.), Improving student learning: Theory and practice ten years on (pp. 412–424). Oxford Centre for Staff and Learning Development, Oxford Brookes University.

This is a foundational source for using liminality as a lens on learners' transition through troublesome knowledge. (University of Edinburgh record)

Computational Thinking

Wing, J. M. (2006). Computational thinking. Communications of the ACM, 49(3), 33–35. https://doi.org/10.1145/1118178.1118215

Grounds the computational practices students use to represent and reason through problems. (ACM publication)

Reflective inquiry

Dewey, J. (1910). How we think. D. C. Heath & Co.

Supports the role of curiosity, inquiry, evidence, and reflection in learning. (Digitised text)

Learning and transfer

Bransford, J. D., Brown, A. L., & Cocking, R. R. (Eds.). (2000). How people learn: Brain, mind, experience, and school: Expanded edition. National Academies Press. https://doi.org/10.17226/9853

A useful foundation for prior knowledge, learning environments, and how students use knowledge in new situations. (National Academies Press)

~*~*~*~*~*~